SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: ITA 06 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 20%

QUESTIONS:03 **Total Marks:25**

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I **Jeevan Raaja S (192421005)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	*Conceptual Understanding*	*Accuracy of Answers*	*Application of Knowledge*	*Completeness of Response*	*Clarity & Presentation*	*Total*
Weightage	*25%*	*25%*	*20%*	*15%*	*15%*	
Q1						
Q2						
Q3						

Signature of the student: Jeevan Raaja S ***Total Marks :***

Annexure A

1. Create a pseudocode algorithm to detect and handle **outliers in a dataset** using statistical methods such as Z-score or IQR. The algorithm should identify extreme values, decide whether to remove or cap them, and ensure that the resulting dataset maintains consistency and reliability for further analysis.

2. Write a pseudocode algorithm to implement **data aggregation and summarization** across multiple categories. The algorithm should group data based on specified attributes, compute aggregated metrics (sum, average, count), and generate a summarized report. Include steps for sorting, indexing, and validating the correctness of aggregated results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**25**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Pseudocode Algorithm to Detect and Handle Outliers

Aim

To design an algorithm that detects extreme values using **Z-score or IQR**, decides whether to remove or cap the outliers, and produces a consistent and reliable dataset.

Pseudocode

```
ALGORITHM Detect_And_Handle_Outliers

INPUT: Dataset D
OUTPUT: Cleaned Dataset D_clean

BEGIN

1. LOAD dataset D

2. SELECT numerical column X from D

3. CHECK for missing values in X
   IF missing values exist THEN
      HANDLE missing values using an appropriate method
   END IF

4. CALCULATE Q1 = 25th percentile of X
5. CALCULATE Q3 = 75th percentile of X
6. CALCULATE IQR = Q3 - Q1

7. CALCULATE Lower_Bound = Q1 - 1.5 × IQR
8. CALCULATE Upper_Bound = Q3 + 1.5 × IQR

9. FOR each value x in X DO
      IF x < Lower_Bound OR x > Upper_Bound THEN
         MARK x as OUTLIER
      ELSE
         MARK x as NORMAL
```

```
    END IF
  END FOR

10. COUNT the number of detected outliers

11. FOR each detected OUTLIER DO
      IF outlier is a valid extreme observation THEN
        CAP the value:
          IF x < Lower_Bound THEN
            x = Lower_Bound
          ELSE IF x > Upper_Bound THEN
            x = Upper_Bound
          END IF
      ELSE
        REMOVE the outlier record
      END IF
    END FOR

12. STORE the resulting data as D_clean

13. VALIDATE D_clean
      CHECK for missing values
      CHECK minimum and maximum values
      CHECK that values are within acceptable bounds

14. COMPARE D and D_clean
      CHECK number of records
      CHECK number of outliers handled
      CHECK statistical consistency

15. RETURN D_clean

END
```

Explanation

The **IQR method** detects outliers using the first quartile (Q1) and third quartile (Q3). The IQR is calculated as:

IQR = Q3 − Q1

The acceptable range is:

Lower Bound = Q1 − 1.5 × IQR

Upper Bound = Q3 + 1.5 × IQR

Any value below the lower bound or above the upper bound is identified as an outlier.

Instead of automatically deleting every extreme value, the algorithm makes a decision. If the value is a genuine but extreme observation, it can be **capped** at the appropriate boundary. If it is an invalid or erroneous observation, it can be **removed**.

Finally, the cleaned dataset is validated to ensure that the data remains consistent and reliable for further analysis.

Example

Suppose the values are:

10, 12, 11, 13, 12, 100

Here, `100` is much larger than the other observations and may be identified as an outlier.

After applying the IQR method, the algorithm can either:

- **Remove** `100` if it is an incorrect measurement, or
- **Cap** it at the calculated upper bound if it is a valid extreme observation.

Result

The algorithm produces a **cleaned dataset with extreme values appropriately handled**, improving the consistency and reliability of subsequent data analysis.

Q2. Pseudocode for Data Aggregation and Summarization

Aim

To design an algorithm that groups data according to specified attributes, calculates **sum, average, and count**, sorts and indexes the results, and validates the correctness of the aggregated report.

Pseudocode

```
ALGORITHM Aggregate_And_Summarize

INPUT: Dataset D
    Grouping_Attribute G
    Numerical_Attribute V

OUTPUT: Summary_Report

BEGIN

1. LOAD dataset D

2. CHECK whether G and V exist in D
  IF required attributes do not exist THEN
    DISPLAY "Invalid attributes"
    STOP
  END IF
```

3. CHECK dataset for missing or invalid values

4. HANDLE missing values appropriately

5. GROUP records in D according to attribute G

6. FOR each group C DO

    SET Count = number of records in C

    SET Sum = sum of values of V in C

    SET Average = Sum / Count

    STORE:

      Group_Name

      Count

      Sum

      Average

  END FOR

7. CREATE Summary_Table using:

    Group_Name

    Count

    Sum

    Average

8. SORT Summary_Table by Average in descending order

9. CREATE an index for Summary_Table

10. DISPLAY Summary_Table

11. VALIDATE aggregated results

12. CALCULATE:

    Total_Count = sum of all group counts

    Total_Sum = sum of all group sums

13. COMPARE:

    Total_Count with original valid record count

    Total_Sum with original valid data sum

14. IF validation is successful THEN

    DISPLAY "Aggregation is correct"

  ELSE

    DISPLAY "Aggregation error detected"

  END IF

15. GENERATE final Summary_Report

16. RETURN Summary_Report

END

Explanation

The algorithm first loads the dataset and checks whether the required grouping and numerical attributes are available.

For example, if a sales dataset contains:

Category	**Sales**
A	100
B	200
A	150

B	300
C	250

The algorithm groups the records by **Category**.

The aggregation results are:

Category	Count	Sum	Average
A	2	250	125
B	2	500	250
C	1	250	250

The summarized table can then be **sorted** by average:

B → Average = 250

C → Average = 250

A → Average = 125

An index is assigned to make the summarized results easier to access and analyze.

Validation

Validation ensures that aggregation has not lost or duplicated data.

Total_Count = Sum of all group counts

Total_Sum = Sum of all group sums

These values are compared with the corresponding values from the original dataset.

If:

Total_Count = Original Valid Record Count

AND

Total_Sum = Original Valid Data Sum

then the aggregation is considered correct.

Result

The algorithm generates a **sorted and indexed summary report** containing the **count, sum, and average for each category**, while validation ensures that the aggregated results are accurate and complete.